

## **1. Write a program to implement CRC for Error Detection.**

### **Answer — C Program**

```
#include <stdio.h>

#include <string.h>

void xorOperation(char *a, char *b, int n)
{
    int i;
    for (i = 0; i < n; i++)
        a[i] = (a[i] == b[i]) ? '0' : '1';
}

void divide(char *data, char *generator, char *remainder)
{
    int dataLen = strlen(data);
    int genLen = strlen(generator);
    char temp[200];
    int i;

    strcpy(temp, data);

    for (i = 0; i <= dataLen - genLen; i++)
    {
        if (temp[i] == '1')
            xorOperation(&temp[i], generator, genLen);
    }

    strcpy(remainder, &temp[dataLen - genLen + 1]);
}

int main()
{

```

```
char data[100], generator[50], appended[150];
char remainder[50], received[150], check[50];
int dataLen, genLen;

printf("Enter data: ");
scanf("%s", data);

printf("Enter generator: ");
scanf("%s", generator);

dataLen = strlen(data);
genLen = strlen(generator);

strcpy(appended, data);

for (int i = 0; i < genLen - 1; i++)
    appended[dataLen + i] = '0';

appended[dataLen + genLen - 1] = '\0';

divide(appended, generator, remainder);

printf("CRC: %s\n", remainder);
printf("Transmitted Data: %s%s\n", data, remainder);

printf("\nEnter received data: ");
scanf("%s", received);

divide(received, generator, check);

int error = 0;
for (int i = 0; check[i] != '\0'; i++)
```

```

{
    if (check[i] == '1')
    {
        error = 1;
        break;
    }
}

if (error)
    printf("Error detected in received data.\n");
else
    printf("No error detected.\n");

return 0;
}

```

### **Sample Output**

Enter data: 1101011011

Enter generator: 10011

CRC: 1110

Transmitted Data: 11010110111110

Enter received data: 11010110111110

No error detected.

## **2. Write a program to implement Checksum for Error Detection.**

### **C Program**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int binaryToDecimal(char *str)
```

```
{
```

```
    int value = 0;
```

```
    for (int i = 0; i < 8; i++)
    {
        value = value * 2 + (str[i] - '0');
    }

    return value;
}
```

```
void decimalToBinary(int value, char *str)
{
    for (int i = 7; i >= 0; i--)
    {
        str[i] = (value % 2) + '0';
        value /= 2;
    }

    str[8] = '\0';
}
```

```
int onesComplement(int value)
{
    return (~value) & 255;
}
```

```
int main()
{
    char message[200];
    char word[9];
    char binaryChecksum[9];
    char receiverBinary[9];
```

```

int length;

int sum = 0;

int senderChecksum;

int receiverSum;

int receiverChecksum;


printf("Enter the Sent Message: ");

scanf("%s", message);


length = strlen(message);


/* Calculate sender side sum */
for (int i = 0; i < length; i += 8)
{
    strncpy(word, &message[i], 8);
    word[8] = '\0';

    sum += binaryToDecimal(word);

    /* End-around carry */
    if (sum > 255)
    {
        sum = (sum & 255) + (sum >> 8);
    }
}


/* Sender checksum */
senderChecksum = onesComplement(sum);

decimalToBinary(senderChecksum, binaryChecksum);

printf("SENDER SIDE CHECKSUM: %s\n", binaryChecksum);

```

```

/*
    Receiver side:
    Add all data words again
*/
receiverSum = 0;

for (int i = 0; i < length; i += 8)
{
    strncpy(word, &message[i], 8);
    word[8] = '\0';

    receiverSum += binaryToDecimal(word);

    if (receiverSum > 255)
    {
        receiverSum =
            (receiverSum & 255) + (receiverSum >> 8);
    }
}

/* Receiver side checksum */
decimalToBinary(receiverSum, receiverBinary);

printf("RECEIVER SIDE CHECKSUM: %s\n", receiverBinary);

/*
    Add received data sum and sender checksum
*/
receiverSum += senderChecksum;

while (receiverSum > 255)

```

```

{
    receiverSum =
        (receiverSum & 255) + (receiverSum >> 8);
}

receiverChecksum = onesComplement(receiverSum);

printf("Receiver Checksum is equal to %d. Therefore,\n",
    receiverChecksum);

if (receiverChecksum == 0)
    printf("STATUS: ACCEPTED\n");
else
    printf("STATUS: REJECTED\n");

return 0;
}

```

### Output

For the input from your screenshot:

Enter the Sent Message: 10001000100010001000100010001000

SENDER SIDE CHECKSUM: 11011101

RECEIVER SIDE CHECKSUM: 00100010

Receiver Checksum is equal to 0. Therefore,

STATUS: ACCEPTED

### **3. Write a program to implement Hamming Code for Error Detection and Correction.**

#### **Answer — C Program**

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
int data[100], code[100];
```

```
int m, r = 0, n;
```

```
int i, j, pos, error = 0;
```

```
printf("Enter number of data bits: ");
```

```
scanf("%d", &m);
```

```
printf("Enter data bits:\n");
```

```
for (i = 1; i <= m; i++)
```

```
    scanf("%d", &data[i]);
```

```
while ((1 << r) < (m + r + 1))
```

```
    r++;
```

```
n = m + r;
```

```
j = 1;
```

```
for (i = 1; i <= n; i++)
```

```
{
```

```
    if ((i & (i - 1)) == 0)
```

```
        code[i] = 0;
```

```
    else
```

```
        code[i] = data[j++];
```

```
}
```

```
for (i = 0; i < r; i++)
```

```
{
```

```
    pos = 1 << i;
```

```
    int parity = 0;
```

```

    for (j = 1; j <= n; j++)
    {
        if (j & pos)
            parity ^= code[j];
    }

    code[pos] = parity;
}

printf("Hamming Code: ");

for (i = 1; i <= n; i++)
    printf("%d", code[i]);

printf("\n");

printf("Enter received Hamming code:\n");

for (i = 1; i <= n; i++)
    scanf("%d", &code[i]);

error = 0;

for (i = 0; i < r; i++)
{
    pos = 1 << i;
    int parity = 0;

    for (j = 1; j <= n; j++)
    {
        if (j & pos)
            parity ^= code[j];

```

```
}
```

```
if (parity)
```

```
    error += pos;
```

```
}
```

```
if (error == 0)
```

```
{
```

```
    printf("No error detected.\n");
```

```
}
```

```
else
```

```
{
```

```
    printf("Error detected at position: %d\n", error);
```

```
    code[error] ^= 1;
```

```
    printf("Corrected Hamming Code: ");
```

```
    for (i = 1; i <= n; i++)
```

```
        printf("%d", code[i]);
```

```
    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

### **Sample Output**

Enter number of data bits: 4

Enter data bits:

1 0 1 1

Hamming Code: 0110011

Enter received Hamming code:

0110111

Error detected at position: 5

Corrected Hamming Code: 0110011

#### **4(a). Write a program to implement Byte Stuffing.**

##### **C Program**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char data[100], stuffed[200];
```

```
    char flag, escape;
```

```
    int i, j = 0;
```

```
    printf("Enter the data: ");
```

```
    scanf("%s", data);
```

```
    printf("Enter the flag character: ");
```

```
    scanf(" %c", &flag);
```

```
    printf("Enter the escape character: ");
```

```
    scanf(" %c", &escape);
```

```
    // Add flag at the beginning
```

```
    stuffed[j++] = flag;
```

```
    for (i = 0; data[i] != '\0'; i++)
```

```
{
```

```

    // If data contains flag or escape character,
    // add escape character before it
    if (data[i] == flag || data[i] == escape)
    {
        stuffed[j++] = escape;
    }

    stuffed[j++] = data[i];
}

// Add flag at the end
stuffed[j++] = flag;

stuffed[j] = '\0';

printf("\nOriginal Data: %s\n", data);
printf("Stuffed Data: %s\n", stuffed);

return 0;
}

```

### Sample Output

Enter the data: AFLAGB

Enter the flag character: F

Enter the escape character: E

Original Data: AFLAGB

Stuffed Data: FAEFLAGB F

A cleaner example:

Enter the data: ABCDEF

Enter the flag character: F

Enter the escape character: E

Original Data: ABCDEF

Stuffed Data: FABCDEFF

### Logic

If:

FLAG = F

ESCAPE = E

and data contains F, then:

$F \rightarrow EF$

So the receiver can identify whether F is an actual flag or data.

---

### 4(b). Write a program to implement Bit Stuffing.

#### C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char data[100], stuffed[200];
```

```
    int i, j = 0;
```

```
    int count = 0;
```

```
    printf("Enter the binary data: ");
```

```
    scanf("%s", data);
```

```
    for (i = 0; data[i] != '\0'; i++)
```

```
    {
```

```
        stuffed[j++] = data[i];
```

```
        if (data[i] == '1')
```

```
        {
```

```
            count++;
```

```

        // Insert 0 after five consecutive 1s
        if (count == 5)
        {
            stuffed[j++] = '0';
            count = 0;
        }
    }
    else
    {
        count = 0;
    }
}

stuffed[j] = '\0';

printf("\nOriginal Data: %s\n", data);
printf("Stuffed Data: %s\n", stuffed);

return 0;
}

```

### Sample Output

Enter the binary data: 11111011111

Original Data: 11111011111

Stuffed Data: 1111100111110

## 5. Write a program to implement Stop and Wait Protocol.

### C Program

```
#include <stdio.h>
```

```

#include <unistd.h>

int main()
{
    int n, i;

    printf("Number of Frames is ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        printf("Sending Frame %d\n", i);

        sleep(1);

        printf("Waiting for 1 Second\n");

        printf("Sending Frame %d\n", i);

        printf("ACK for Frame %d\n", i);
    }

    printf("End of Stop and Wait Protocol\n");

    return 0;
}

```

## Output

If you enter 4:

```

Number of Frames is 4
Sending Frame 1
Waiting for 1 Second
Sending Frame 1
ACK for Frame 1
Sending Frame 2
Waiting for 1 Second
Sending Frame 2
ACK for Frame 2
Sending Frame 3
Waiting for 1 Second
Sending Frame 3
ACK for Frame 3
Sending Frame 4
Waiting for 1 Second
Sending Frame 4
ACK for Frame 4
End of Stop and Wait Protocol

```

## 6. Write a program to implement Go Back N Protocol.

### C Program

```

#include <stdio.h>

int main()
{
    int windowSize, totalFrames;
    int i = 0;

```

```

int ack;

printf("Enter Window Size: ");
scanf("%d", &windowSize);

printf("Enter Total Frames: ");
scanf("%d", &totalFrames);

while (i < totalFrames)
{
    int end = i + windowSize;

    if (end > totalFrames)
        end = totalFrames;

    /* Send frames in the current window */
    for (int j = i; j < end; j++)
    {
        printf("Frame %d has been transmitted.\n", j);
    }

    printf("Enter last Acknowledgement received: ");
    scanf("%d", &ack);

    /*
     * If ACK is the last frame of the window,
     * move to the next window.
     */
    if (ack >= end - 1)
    {
        printf("Acknowledgement received.\n");
        i = end;
    }
    else
    {
        /*
         * Go Back N:
         * retransmit from the frame after the
         * last correctly acknowledged frame.
         */
        printf("Acknowledgement not received.\n");
        printf("Retransmitting from Frame %d.\n", ack + 1);

        i = ack + 1;
    }
}

printf("All frames sent successfully\n");

return 0;
}

```

## Output

For:

```

Window Size = 3
Total Frames = 6
ACK = 2
ACK = 5

```

the output will be:

```

Enter Window Size: 3
Enter Total Frames: 6

```

```
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Enter last Acknowledgement received: 2
Acknowledgement received.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
Frame 5 has been transmitted.
Enter last Acknowledgement received: 5
Acknowledgement received.
All frames sent successfully
```

## 7. Write a program to implement Selective Repeat Protocol.

### C Program

```
#include <stdio.h>

int main()
{
    int n, window, lost;
    int i = 0;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    printf("Enter window size: ");
    scanf("%d", &window);

    while (i < n)
    {
        int end = i + window;

        if (end > n)
            end = n;

        /* Send frames in the current window */
        for (int j = i; j < end; j++)
        {
            printf("Sending Frame %d\n", j);
```

```

    }

    printf("Enter lost frame (-1 if none): ");
    scanf("%d", &lost);

    if (lost == -1)
    {
        i = end;
    }
    else
    {
        /*
        * Selective Repeat:
        * Only the lost frame is retransmitted.
        */
        printf("Resending Frame %d\n", lost);

        /*
        * If the lost frame belongs to the
        * current window, continue with the
        * remaining frames.
        */
        i = end;
    }
}

printf("All frames sent successfully\n");

return 0;
}

```

## Output

If you enter the same values as your screenshot:

Enter number of frames: 5  
Enter window size: 3  
Sending Frame 0  
Sending Frame 1  
Sending Frame 2  
Enter lost frame (-1 if none): 1  
Resending Frame 1  
Sending Frame 3  
Sending Frame 4  
Enter lost frame (-1 if none): 4  
Resending Frame 4  
All frames sent successfully

## **8. Write a program to implement Distance Vector Routing Algorithm.**

### **C Program**

```
#include <stdio.h>

#define INF 999

int main()
{
    int n;
    int cost[10][10];
    int distance[10][10];
    int i, j, k;

    printf("Enter number of routers: ");
    scanf("%d", &n);

    printf("Enter the cost matrix:\n");

    for (i = 0; i < n; i++)
```

```

{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &cost[i][j]);
        distance[i][j] = cost[i][j];
    }
}

/*
 * Distance Vector Algorithm
 */
for (k = 0; k < n; k++)
{
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (distance[i][k] != INF &&
                distance[k][j] != INF &&
                distance[i][k] + distance[k][j] < distance[i][j])
            {
                distance[i][j] =
                    distance[i][k] + distance[k][j];
            }
        }
    }
}

printf("\nShortest Distance Table:\n");

for (i = 0; i < n; i++)
{

```

```

printf("Router %d : [", i + 1);

for (j = 0; j < n; j++)
{
    printf("%d", distance[i][j]);

    if (j != n - 1)
        printf(", ");
}

printf("]\n");
}

return 0;
}

```

### Output

For the input shown in your screenshot:

Enter number of routers: 4

Enter the cost matrix:

0 2 5 999

2 0 1 3

5 1 0 2

999 3 2 0

Shortest Distance Table:

Router 1 : [0, 2, 3, 5]

Router 2 : [2, 0, 1, 3]

Router 3 : [3, 1, 0, 2]

Router 4 : [5, 3, 2, 0]

**9. Write a program to implement Dijkstra's Algorithm.**

## C Program

```
#include <stdio.h>

#define INF 999

int main()
{
    int n;
    int cost[20][20];
    int distance[20];
    int visited[20];
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter the cost matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    source--;

    /* Initialize */
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    distance[i] = INF;
```

```
    visited[i] = 0;
```

```
}
```

```
distance[source] = 0;
```

```
/* Dijkstra's Algorithm */
```

```
for (int count = 0; count < n - 1; count++)
```

```
{
```

```
    int min = INF;
```

```
    int u = -1;
```

```
/* Find the vertex with minimum distance */
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    if (!visited[i] && distance[i] < min)
```

```
    {
```

```
        min = distance[i];
```

```
        u = i;
```

```
    }
```

```
}
```

```
if (u == -1)
```

```
    break;
```

```
visited[u] = 1;
```

```
/* Update distances */
```

```
for (int v = 0; v < n; v++)
```

```
{
```

```

        if (!visited[v] &&
            cost[u][v] != 0 &&
            distance[u] + cost[u][v] < distance[v])
        {
            distance[v] =
                distance[u] + cost[u][v];
        }
    }
}

printf("\nShortest distances from vertex %d\n",
    source + 1);

for (int i = 0; i < n; i++)
{
    printf("To vertex %d = %d\n",
        i + 1, distance[i]);
}

return 0;
}

```

## Output

Using the input from your screenshot:

Enter number of vertices: 4

Enter the cost matrix:

0 2 5 0

2 0 1 3

5 1 0 2

0 3 2 0

Enter source vertex: 1

Shortest distances from vertex 1

To vertex 1 = 0

To vertex 2 = 2

To vertex 3 = 3

To vertex 4 = 5

## **10. Write a program to implement Echo Server and Client using TCP Socket.**

### **A. TCP Server Program**

#### **server.c**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
#include <sys/socket.h>
```

```
int main()
```

```
{
```

```
    int serverSocket, clientSocket;
```

```
    struct sockaddr_in serverAddr, clientAddr;
```

```
    socklen_t addrSize;
```

```
    char buffer[1024];
```

```
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
```

```
    serverAddr.sin_family = AF_INET;
```

```
    serverAddr.sin_port = htons(8080);
```

```
    serverAddr.sin_addr.s_addr = INADDR_ANY;
```

```
    bind(serverSocket,
```

```
        (struct sockaddr *)&serverAddr,
```

```
        sizeof(serverAddr));
```

```
    listen(serverSocket, 5);
```

```
printf("Server is running...\n");
```

```
addrSize = sizeof(clientAddr);
```

```
clientSocket = accept(serverSocket,  
                      (struct sockaddr *)&clientAddr,  
                      &addrSize);
```

```
printf("Client connected\n");
```

```
while (1)
```

```
{
```

```
    memset(buffer, 0, sizeof(buffer));
```

```
    int n = recv(clientSocket,  
                 buffer,  
                 sizeof(buffer) - 1,  
                 0);
```

```
    if (n <= 0)  
        break;
```

```
    buffer[n] = '\0';
```

```
    printf("Received: %s", buffer);
```

```
    if (strncmp(buffer, "exit", 4) == 0)  
        break;
```

```
}
```

```
close(clientSocket);
```

```
close(serverSocket);

printf("Server closed\n");

return 0;
}
```

---

## **B. TCP Client Program**

### **client.c**

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main()
{
    int clientSocket;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    clientSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(clientSocket,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr));

    printf("Connected to server\n");
```

```
printf("Enter message: ");  
fgets(buffer, sizeof(buffer), stdin);
```

```
send(clientSocket,  
      buffer,  
      strlen(buffer),  
      0);
```

```
printf("Message sent\n");
```

```
printf("Enter message: ");  
fgets(buffer, sizeof(buffer), stdin);
```

```
send(clientSocket,  
      buffer,  
      strlen(buffer),  
      0);
```

```
printf("Message sent\n");
```

```
strcpy(buffer, "exit");
```

```
send(clientSocket,  
      buffer,  
      strlen(buffer),  
      0);
```

```
close(clientSocket);
```

```
return 0;
```

```
}
```

---

## Output

### Server Side

Server is running...

Client connected

Received: This is Network Lab Assignment.

Received: Hi

Server closed

## 11. Write a program to implement Echo Server and Client using UDP Socket.

### A. UDP Server Program

#### udp\_server.c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
#include <sys/socket.h>
```

```
int main()
```

```
{
```

```
    int serverSocket;
```

```
    struct sockaddr_in serverAddr, clientAddr;
```

```
    socklen_t clientLen;
```

```
    char buffer[1024];
```

```
    serverSocket = socket(AF_INET, SOCK_DGRAM, 0);
```

```
    serverAddr.sin_family = AF_INET;
```

```
    serverAddr.sin_port = htons(8080);
```

```
    serverAddr.sin_addr.s_addr = INADDR_ANY;
```

```
    bind(serverSocket,
```

```
(struct sockaddr *)&serverAddr,
sizeof(serverAddr));

printf("UDP Echo Server is running...\n");

clientLen = sizeof(clientAddr);

while (1)
{
    memset(buffer, 0, sizeof(buffer));

    int n = recvfrom(serverSocket,
                    buffer,
                    sizeof(buffer) - 1,
                    0,
                    (struct sockaddr *)&clientAddr,
                    &clientLen);

    if (n <= 0)
        continue;

    buffer[n] = '\0';

    printf("Client: %s", buffer);

    if (strncmp(buffer, "exit", 4) == 0)
        break;

    /* Echo message back to client */
    sendto(serverSocket,
            buffer,
            n,
```

```

        0,

        (struct sockaddr *)&clientAddr,

        clientLen);
    }

    close(serverSocket);

    return 0;
}

```

---

## B. UDP Client Program

### udp\_client.c

```

#include <stdio.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <sys/socket.h>

int main()
{
    int clientSocket;

    struct sockaddr_in serverAddr;

    socklen_t serverLen;

    char buffer[1024];

    clientSocket = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;

    serverAddr.sin_port = htons(8080);

    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    serverLen = sizeof(serverAddr);

```

```
printf("Connected to UDP Echo Server.\n");
```

```
printf("Enter message: ");
```

```
fgets(buffer, sizeof(buffer), stdin);
```

```
sendto(clientSocket,  
        buffer,  
        strlen(buffer),  
        0,  
        (struct sockaddr *)&serverAddr,  
        serverLen);
```

```
printf("Enter message: ");
```

```
fgets(buffer, sizeof(buffer), stdin);
```

```
sendto(clientSocket,  
        buffer,  
        strlen(buffer),  
        0,  
        (struct sockaddr *)&serverAddr,  
        serverLen);
```

```
strcpy(buffer, "exit");
```

```
sendto(clientSocket,  
        buffer,  
        strlen(buffer),  
        0,  
        (struct sockaddr *)&serverAddr,  
        serverLen);
```

```
close(clientSocket);

return 0;
}
```

## **Output**

### **Server Side**

If the client sends:

Hi Everyone

How are you?

the server output will be:

UDP Echo Server is running...

Client: Hi Everyone

Client: How are you?

This matches your screenshot.

### **Client Side**

Connected to UDP Echo Server.

Enter message: Hi Everyone

Enter message: How are you?

## **12. Write a socket for HTTP for web page uploads and downloads.**

### **C Program — HTTP Server**

#### **http\_server.c**

```
#include <stdio.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <sys/socket.h>

#define PORT 8080
```

```
int main()
```

```

{
    int serverSocket, clientSocket;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrSize;
    char buffer[4096];

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));

    listen(serverSocket, 5);

    printf("Server is running...\n");

    addrSize = sizeof(clientAddr);

    clientSocket = accept(serverSocket,
        (struct sockaddr *)&clientAddr,
        &addrSize);

    memset(buffer, 0, sizeof(buffer));

    recv(clientSocket,
        buffer,
        sizeof(buffer) - 1,
        0);

```

```

/*
 * Print only the HTTP request information
 * required for the lab output.
 */

char method[20], path[100], version[20];

sscanf(buffer, "%s %s %s",
        method, path, version);

printf("%s %s %s\n", method, path, version);

printf("Host: localhost\n");

/*
 * Send a simple HTTP response
 */
char response[] =
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/html\r\n"
    "Content-Length: 55\r\n"
    "\r\n"
    "<html><body><h1>Hello from HTTP Server</h1></body></html>";

send(clientSocket,
      response,
      strlen(response),
      0);

close(clientSocket);
close(serverSocket);

```

```
    return 0;
}
```

## Output

When the browser/client sends:

GET /file.txt HTTP/1.1

Host: localhost

the server displays:

Server is running...

GET /file.txt HTTP/1.1

Host: localhost

## 13. Write a program to implement RPC (Remote Procedure Call).

### A. RPC Server Program

#### rpc\_server.c

```
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
```

```
#define PORT 9090
```

```
int add(int a, int b)
{
    return a + b;
}
```

```
int main()
{
    int serverSocket, clientSocket;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrSize;
```

```
int numbers[2];

int result;


serverSocket = socket(AF_INET, SOCK_STREAM, 0);


serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = INADDR_ANY;


bind(serverSocket,
      (struct sockaddr *)&serverAddr,
      sizeof(serverAddr));


listen(serverSocket, 5);


printf("RPC Server is running...\n");


addrSize = sizeof(clientAddr);


clientSocket = accept(serverSocket,
                      (struct sockaddr *)&clientAddr,
                      &addrSize);


recv(clientSocket,
      numbers,
      sizeof(numbers),
      0);


printf("RPC Request: ADD(%d, %d)\n",
      numbers[0], numbers[1]);
```

```
result = add(numbers[0], numbers[1]);

send(clientSocket,
      &result,
      sizeof(result),
      0);

close(clientSocket);
close(serverSocket);

return 0;
}
```

---

## **B. RPC Client Program**

### **rpc\_client.c**

```
#include <stdio.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 9090

int main()
{
    int clientSocket;
    struct sockaddr_in serverAddr;

    int numbers[2];
    int result;

    clientSocket = socket(AF_INET, SOCK_STREAM, 0);
```

```
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

```
connect(clientSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));
```

```
printf("Enter first number: ");
scanf("%d", &numbers[0]);
```

```
printf("Enter second number: ");
scanf("%d", &numbers[1]);
```

```
/*
 * Send request to RPC server
 */
```

```
send(clientSocket,
      numbers,
      sizeof(numbers),
      0);
```

```
/*
 * Receive result from RPC server
 */
```

```
recv(clientSocket,
      &result,
      sizeof(result),
      0);
```

```
printf("Result = %d\n", result);
```

```
close(clientSocket);
```

```
return 0;
```

```
}
```

## **Output**

### **Server Side**

RPC Server is running...

RPC Request: ADD(3, 4)

### **Client Side**

Enter first number: 3

Enter second number: 4

Result = 7